

Razvoj web aplikacija

Predavanje 1: Arhitektura backenda i kostur projekta

doc. dr. sc. Tomislav Petković mag. ing. Josip Torić

08.10.2026.



Plan predavanja

- ➊ Od zahtjeva do odgovora: što backend radi i kako se dijeli u slojeve
- ➋ Ugovor prema klijentu: rute, statusni kodovi, sheme
- ➌ Kostur projekta: struktura, konfiguracija, pokretanje
- ➍ Dijagnostika i kvaliteta: zapisnik, pogreške, statička analiza
- ➎ Rad s verzijama koda

Na kraju predavanja projekt je pokrenut: baza u kontejneru, backend koji odgovara na zahtjev i zapisan postupak pokretanja.

Ishodi učenja

Nakon ovog predavanja student može:

- objasniti put HTTP zahtjeva kroz slojeve backenda
- razgraničiti odgovornosti slojeva routers, services i repositories
- postaviti razvojnu okolinu: baza u kontejneru i konfiguracija iz varijabli okoline
- objasniti kako se aplikacija sastavlja i pokreće
- čitati promjene u kodu naredbom `git diff`

Domena primjera

Sustav prijava klubova na powerlifting natjecanje.

- Klub prijavljuje svoje natjecatelje i vidi samo svoje prijave; organizator vidi prijave svih klubova.
- Natjecanje ima dva roka nominacije. Do prvoga roka prijave se dodaju, mijenjaju i povlače. Između rokova mogu se još mijenjati i povlačiti, ali ne i dodavati. Nakon drugoga roka popis je zaključan.

Ista se struktura pojavljuje u mnogim sustavima: korisnik vidi dio podataka, a operacije nad njima ovise o tome u kojem je stanju nadređeni zapis.

Od zahtjeva do odgovora

Što putuje mrežom, jednostavan backend, obrada zahtjeva, i sloj po sloj koji je obavlja.

HTTP zahtjev i odgovor

Zahtjev je tekst: metoda, putanja, zaglavlja i, po potrebi, tijelo.

```
GET /clubs/12/lifters HTTP/1.1
Host: api.primjer.hr
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

Odgovor je također tekst: statusni kod, zaglavlja i tijelo.

```
HTTP/1.1 200 OK
Content-Type: application/json

[{"id": 31, "ime": "Ana Horvat"}]
```

Backend je program koji prima prvi tekst i sastavlja drugi.

Primjer jednostavnog backenda

```
from fastapi import APIRouter

router = APIRouter()

@router.get("/competitions")
def list_competitions():
    return [
        {"id": 7, "naziv": "Zimski kup", "datum": "2027-03-14"},
        {"id": 8, "naziv": "Ljetni kup", "datum": "2027-06-06"},
    ]
```

Framework (ovdje FastAPI) povezuje par metode i putanje s funkcijom. Funkcija vraća podatke, a framework ih pretvara u JSON, dodaje zaglavlja i statusni kod 200. Funkcija pridružena putanji naziva se **ruta**.

Obrada zahtjeva

Popis natjecatelja jednoga kluba nije javan, a prijava natjecatelja k tome mijenja podatke. Za obradu takvog zahtjeva i sastavljanje odgovora moramo:

- 1 provjeriti oblik i tipove poslanih podataka
- 2 utvrditi tko šalje zahtjev
- 3 provjeriti smije li taj korisnik do tih podataka
- 4 provjeriti dopuštaju li poslovna pravila izmjenu u ovom trenutku
- 5 upisati promjenu tako da pri pogrešci ne ostane polovičan trag

Svaki zahtjev koji mijenja podatke prolazi kroz ove korake.

Od koraka do slojeva

Koraci su različiti poslovi, ali ih aplikacija ne obavlja jednom: svaka ruta koja dira iste podatke prolazi kroz iste korake.

Slojevi su odgovor na pitanje **gdje u kodu svaki korak stoji**. Svaki korak dobiva jedno mjesto, a rute ga odande pozivaju umjesto da ga same izvode.

- koraci 2 i 6 (primanje zahtjeva i sastavljanje odgovora): routers
- korak 1 (oblik i tipovi poslanih podataka): schemas
- koraci 3 i 4 (prava pristupa i poslovna pravila): services
- korak 5 (upis u bazu): repositories

Sloj ruta: routers

```
# app/routers/registrations.py

@router.post("/{competition_id}/registrations", status_code=201)
def create_registration(
    competition_id: int,
    data: RegistrationCreate,
    user: User = Depends(get_current_user),
):
    return registration_service.create(competition_id, data, user)
```

Dobro napisana ruta je minimalna. Iz putanje uzima broj natjecanja, iz tijela zahtjeva već provjerene podatke, iz tokena korisnika, i sve to predaje sloju services. Njezin je cijeli sadržaj prihvaćanje HTTP-a: putanja, metoda, statusni kod.

Ruta bez sloja ispod sebe

Rute ne moraju pozivati sloj services, na primjer:

```
# app/routers/health.py

@router.get("/")
def health():
    return {"status": "ok"}
```

Odgovor na GET /health potvrđuje troje: proces radi, usmjeravanje zahtjeva radi i aplikacija se uspjela sastaviti.

Tu rutu ne poziva korisnik nego nadzorni alat: Docker je zove pri provjeri stanja kontejnera, a platforme za postavljanje u produkciju u pravilnim razmacima, da utvrde je li primjerak aplikacije spreman primiti promet. Primjerak koji ne odgovori ispada iz razdiobe prometa.

Sloj shema: schemas

```
# app/schemas/registration.py

class RegistrationCreate(BaseModel):
    lifter_id: int
    weight_class: Literal[59, 66, 74, 83, 93, 105, 120]
```

Schema opisuje što ruta prima. Zahtjev koji joj ne odgovara ne dolazi do funkcije: framework ga odbija i u odgovoru imenuje polje koje nije prošlo.

Ograničenje stoji u tipu polja, pa provjera ne postoji kao zaseban redak koda koji se može zaboraviti. U našem primjeru tip nabraja dopuštene težinske kategorije.

Zašto nam treba sloj schemas

Provjera oblika podataka mogla bi stajati i u ruti, kao niz uvjeta na početku funkcije. Odvojena u shemu, dobiva tri svojstva koja tako napisana provjera nema:

- ista se shema koristi u svakoj ruti koja prima takav zapis, pa se pravila o obliku ne prepisuju
- odgovor o pogrešci ima isti oblik za sva polja i sve rute, jer ga sastavlja framework
- opis ulaza i izlaza postoji kao podatak, iz kojega framework gradi dokumentaciju API-ja

Uz to, provjera se dogodi prije nego funkcija počne raditi, pa poslovna pravila nikada ne primaju podatke krivog tipa.

Sloj poslovnih pravila: services

```
# app/services/registration.py

def create(competition_id, data, user):
    competition = competitions.get(competition_id)
    if competition.prelim_deadline < now():
        raise RegistrationClosed()

    lifter = lifters.get(data.lifter_id)
    if lifter.club_id != user.club_id:
        raise NotOwned()

    return registrations.insert(competition_id, data)
```

U ovom sloju stoje odluke o tome smije li se izmjena izvesti. Funkcija prima obične vrijednosti, a ne zahtjev, i ne piše upite nad bazom. Kad pravilo ne prolazi, podiže iznimku i zaustavlja obradu prije nego išta bude upisano.

Zašto nam treba sloj services

Prijava natjecatelja, izmjena prijave i odjava tri su rute, a sve tri prije izmjene moraju provjeriti je li rok prošao. Bez sloja services iste ili slične provjere moramo napisati u kodu svake od tih ruta:

```
# app/routers/registrations.py

def create_registration(...):
    if competition.prelim_deadline < now(): # first copy
        raise HTTPException(409)

def update_registration(...):
    if competition.prelim_deadline < now(): # second copy
        raise HTTPException(409)

def withdraw_registration(...):
    if competition.prelim_deadline < now(): # third copy
        raise HTTPException(409)
```

Što kad se pravilo promijeni

Organizator javlja da rok istječe u ponoć po lokalnom vremenu, a dosad se uspoređivao s koordiniranim svjetskim vremenom (UTC). Provjeru treba promijeniti.

Bez sloja `services` izmjena se mora napraviti u svakoj ruti, a lako se dogodi da se napravi samo ondje gdje je problem uočen, pa preostale dvije nastave po starom. Razlika se vidi samo u satu oko ponoći, pa testovi nepromijenjenih ruta mogu i dalje prolaziti iako kod nije ispravan.

S jednom funkcijom u sloju `services` izmjena je na jednom mjestu:

```
# app/services/registration.py

def ensure_open(competition):
    if competition.prelim_deadline < now():
        raise RegistrationClosed()
```

Cijena podjele je veći broj datoteka i jedan poziv više po zahtjevu. Isplati se čim isti posao pripada većem broju ruta.

Sloj pristupa bazi: repositories

```
# app/repositories/registration.py

def insert(competition_id, data):
    row = Registration(
        competition_id=competition_id,
        **data.model_dump(),
    )
    session.add(row)
    session.commit()
    return row
```

Ovaj sloj zna kako su podatci pohranjeni i ništa više. Ne odlučuje smije li se izmjena izvesti; u našem primjeru to znači da ne provjerava ni rok ni pripadnost natjecatelja klubu.

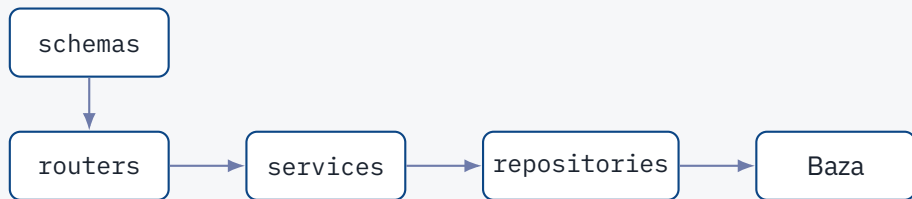
Zašto nam treba sloj repositories

Upit nad bazom mogao bi stajati i u funkciji s pravilima. Odvojen, donosi dvoje:

- **Pravila se čitaju bez baze.** Funkcija s pravilima poziva `insert()` i `get()` umjesto da gradi upite, pa se iz nje vidi koja se odluka donosi, a ne kako su podatci pohranjeni.
- **Promjena pohrane ostaje na jednom mjestu.** Novo polje, promijenjena veza među tablicama ili drukčiji upit zbog brzine mijenjaju samo ovaj sloj.

Isti sloj drži i granice transakcije: niz upisa koji mora proći u cijelosti ili nikako ovdje je jedna cjelina. Da je razasut po pravilima, dio bi zapisa mogao ostati upisan nakon pogreške.

Kako slojevi surađuju



Svaki sloj poziva samo onaj do sebe i nikada onaj iznad: `services` ne zna da postoji `routers`, a `repositories` ne zna da postoje pravila. Rezultat ili iznimka vraćaju se pozivatelju, kao kod svakog poziva funkcije. Iznimku iz sloja `services` u odgovor pretvara handler registriran na razini aplikacije. Zato se svaki sloj može pozvati i bez onih iznad njega: `services` bez pokrenutog poslužitelja, `repositories` bez pravila.

Ugovor prema klijentu

Metoda i putanja, statusni kodovi i njihovi razredi, idempotentnost, sheme.

Metoda i putanja

Putanja imenuje ono nad čime se radi, a metoda kaže što se radi. Par metode i putanje jednoznačno određuje rutu.

Metoda	Putanja	Što poslužitelj radi
GET	/clubs/12/lifters	vraća popis natjecatelja kluba 12
POST	/clubs/12/lifters	dodaje novog natjecatelja u klub 12
GET	/clubs/12/lifters/31	vraća podatke natjecatelja 31
PATCH	/clubs/12/lifters/31	mijenja podatke natjecatelja 31
DELETE	/clubs/12/lifters/31	briše natjecatelja 31

Ista putanja s različitim metodama vodi na različite funkcije. To je svojstvo HTTP-a, a ne izbor aplikacije.

Oblikovanje putanja

HTTP ne propisuje kako putanja treba izgledati. Poslužitelj smije poslušati i `/getLifters` i `/l?id=31`. Ustaljeni oblik postoji zato što ga čitatelj prepoznaje bez dokumentacije.

Konvencija

- Putanja sadrži imenice u množini (`/lifters`), a ne glagole (`/getLifters`). Radnju već nosi metoda.
- Pojedinačni zapis dobiva se dodavanjem oznake (`/lifters/31`).
- Pripadnost se izražava ugnježđivanjem (`/clubs/12/lifters`).

Aplikacija koja odstupa od toga radi jednako dobro. Cijena odstupanja plaća se u vremenu svakoga tko popis ruta prvi put čita.

Razredi statusnih kodova

Svaki odgovor počinje troznamenkastim statusnim kodom. Standard HTTP-a (RFC 9110) dijeli kodove u pet razreda prema prvoj znamenki:

Razred	Značenje
--------	----------

1xx	obavijest o tijeku obrade, odgovor još slijedi
2xx	zahtjev je primljen, razumljiv i obrađen
3xx	potrebna je daljnja radnja, najčešće odlazak na drugu adresu
4xx	zahtjev je pogrešan ili se ne može ispuniti takav kakav je
5xx	zahtjev je bio ispravan, ali ga poslužitelj nije uspio obraditi

Standard traži da klijent razumije razred svakoga koda, i onda kad sam kod ne poznaje. Kod iz razreda 4xx koji klijent ne prepozna smije obraditi kao 400.

Kodovi koji se javljaju u ovakvoj aplikaciji

Od stotinjak kodova koje standard opisuje, ovakva aplikacija koristi ih desetak:

Kod	Značenje	Kod	Značenje
200	obrađeno, tijelo slijedi	405	metoda nad tom putanjom nije podržana
201	nastao je novi zapis	409	sukob sa stanjem resursa
204	obrađeno, tijela nema	415	oblik sadržaja nije podržan
400	zahtjev je neispravan	422	sadržaj se ne može obraditi
401	prijava nedostaje ili ne vrijedi	500	nepredviđeno stanje u poslužitelju
403	zahtjev je odbijen	503	poslužitelj trenutno ne posluhuje
404	na toj putanji nema ničega		

Razred 3xx u aplikaciji koja vraća JSON gotovo se ne pojavljuje, a 1xx se javlja pri nadogradnji veze, primjerice na WebSocket.

Izbor koda u graničnim slučajevima

Kod 401 odnosi se na prijavu, a ne na prava: standard ga opisuje kao nedostatak valjanih podataka za prijavu, premda mu naziv glasi Unauthorized. Kod 403 znači da je pošiljatelj poznat, a zahtjev ipak odbijen. Ostatak izbora standard ne razrješava, jer njegovi opisi dopuštaju više od jednoga.

Konvencija, razlikuje se od projekta do projekta

- Neispravni podatci u tijelu zahtjeva: FastAPI i Pydantic vraćaju 422, ali je moguć i 400.
- Uspješno brisanje: 204 bez tijela ili 200 s podacima obrisano zapisa.
- Zapis koji postoji, ali ga korisnik ne smije vidjeti: 403 ili 404, ovisno o tome smije li se odati da zapis postoji.

Izbor se donosi jednom i dosljedno provodi kroz cijeli projekt.

Idempotentnost

Metoda je idempotentna ako niz od više jednakih zahtjeva ostavlja poslužitelj u istom stanju kao i jedan takav zahtjev. Standard to svojstvo pridružuje samim metodama:

- GET, HEAD, OPTIONS i TRACE ne mijenjaju stanje, pa su idempotentni.
- PUT i DELETE mijenjaju stanje, ali su idempotentni: zapis se postavlja na zadanu vrijednost, odnosno briše.
- POST i PATCH nisu idempotentni.

Posljedica je praktična: klijent smije ponoviti neuspjeli GET ili DELETE bez razmišljanja. Ponovljeni POST može stvoriti drugi zapis, pa zaštita od toga mora doći iz aplikacije, najčešće ograničenjem jedinstvenosti u bazi.

Shema kao opis ulaza i izlaza

Shema je zapis o tome kakve podatke ruta prima i kakve vraća. Framework je provjerava prije nego zahtjev dođe do poslovnih pravila, pa neispravan sadržaj do njih ne stiže.

Shema provjerava:

- jesu li obavezna polja prisutna
- je li svako polje očekivanog tipa
- zadovoljava li vrijednost ograničenje zapisano uz polje, primjerice popis dopuštenih vrijednosti ili raspon

Poslana težinska kategorija	Ishod
-----------------------------	-------

83, vrijednost s popisa	prolazi dalje
85, nije na popisu	odbijeno, odgovor imenuje to polje
„osamdeset tri”	odbijeno, vrijednost nije broj

Opis API-ja nastaje iz koda

OpenAPI je specifikacija: strojno čitljiv opis API-ja u JSON-u, s popisom ruta, metoda, oblika tijela zahtjeva i odgovora te statusnih kodova. Swagger UI je samo jedan prikazivač tog opisa, ne isto što i OpenAPI.

Taj opis ne piše se ručno. FastAPI ga sastavlja iz istih shema i tipova koji su već napisani, pa ne može zastarjeti neovisno o kodu.

- `/openapi.json` je sam opis
- `/docs` je Swagger UI nad njim
- `/redoc` je drugi prikazivač istog opisa

Korist od prvoga dana: API se isprobava bez pisanja ijednog klijenta, a kad u drugom dijelu kolegija dođe frontend, ugovor između njega i backenda već je zapisan.

Dostupnost opisa u produkciji

```
app = FastAPI(  
    title=config.APP_NAME,  
    docs_url="/docs" if config.ENV == "dev" else None,  
    redoc_url=None,  
)
```

Konvencija

Dokumentacija se u produkciji često gasi, da popis ruta i oblik tijela zahtjeva ne budu javno dostupni. Nije obavezno i mnogi projekti je ostavljaju otvorenom; opis tada služi i onima koji se na API spajaju izvana.

Sam `/openapi.json` gasi se zasebno, postavkom `openapi_url`.

Kostur projekta

Po čemu se dijele mape, sastavljanje aplikacije, konfiguracija, pokretanje.

Po čemu se dijele mape

Struktura mapa je odgovor na pitanje gdje tražiti izmjenu. Dva su uobičajena odgovora:

- **po sloju:** sve sheme na jednom mjestu, sve rute na drugom, svi upiti nad bazom na trećem
- **po značajki:** sve što se tiče prijava na jednom mjestu, sve što se tiče korisnika na drugom

Podjela po sloju lakše podnosi rast broja pravila, jer svako pravilo ima očito mjesto. Podjela po značajki lakše podnosi rast broja međusobno nepovezanih dijelova, jer se svaki može čitati i mijenjati zasebno.

Mi ćemo koristiti podjelu po sloju: u aplikacijama s ulogama, stanjima i rokovima raste broj pravila, a ne broj nepovezanih dijelova.

Struktura repozitorija

```
repo/  
  api/ # backend (FastAPI)  
  web/ # frontend (Vue)  
  docker-compose.yaml # database service  
  .env.example # configuration template  
  .gitignore  
  README.md
```

Izbor, ne pravilo

Backend i frontend ovdje stoje u istom repozitoriju. Promjena koja dira obje strane tada je jedan commit umjesto dvaju usklađenih u dvama repozitorijima. Odvojeni repozitoriji imaju svoje razloge, primjerice različite timove i različit ritam izdanja.

Struktura backenda

```
api/  
  app/  
    main.py # creates the application  
  core/  
    config.py # configuration from the environment  
    deps.py # shared dependencies  
    errors.py # domain errors and handlers  
    logging.py # logging setup  
  routers/  
    health.py  
  schemas/  
  models/  
  services/  
  repositories/  
  tests/  
  requirements.txt
```

Mape prate slojeve s prethodnog dijela predavanja, pa se mjesto izmjene određuje prije nego što se otvori ijedna datoteka.

Konfiguracija iz varijabli okoline

```
# app/core/config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    APP_NAME: str = "Powerlifting Registrations API"
    ENV: str = "dev"
    DATABASE_URL: str
    JWT_SECRET: str = "change-me"

settings = Settings()
```

- Vrijednosti se čitaju na jednome mjestu, pa `os.getenv` ne treba pozivati po kodu.
- `.env` ostaje na računalu i ulazi u `.gitignore`; u repozitorij ide `.env.example` s nazivima varijabli i bezopasnim vrijednostima.
- Prijelaz s razvojne okoline na produkcijsku mijenja vrijednosti varijabli, a ne kod.

Sastavljanje aplikacije

Aplikacija nastaje tako da se frameworku prijave rute, postavke i zajedničke ovisnosti.

```
# app/main.py
from fastapi import FastAPI
from app.core.config import Settings, settings
from app.routers.health import router as health_router

def create_app(config: Settings = settings) -> FastAPI:
    app = FastAPI(title=config.APP_NAME, debug=config.ENV == "dev")
    app.include_router(health_router, prefix="/health", tags=["health"])
    return app

app = create_app()
```

Ovdje se postavke iz prethodnog slajda i upotrebljavaju. Posljednji redak stvara primjerak koji poslužitelj pokreće. Isti se poziv može ponoviti drugdje i s drugim postavkama, primjerice `create_app(test_settings)` u testovima.

Baza u kontejneru

```
services:
  db:
    image: postgres:17
    environment:
      POSTGRES_DB: pl_reg
      POSTGRES_USER: pl_user
      POSTGRES_PASSWORD: pl_pass
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Zapis određuje istu inačicu PostgreSQL-a za sve, neovisno o operacijskom sustavu. Imenovani volumen pgdata čuva podatke između pokretanja; brisanjem volumena baza se vraća u prazno stanje.

Pokretanje i provjera

```
docker compose up -d db # database
```

```
cd api
```

```
uv venv
```

```
source .venv/bin/activate
```

```
uv pip install -r requirements.txt
```

```
uvicorn app.main:app --reload
```

Alat uv objedinjuje stvaranje virtualne okoline i instalaciju paketa; requirements.txt ostaje nepromijenjen.

Provjera: `http://127.0.0.1:8000/health` vraća `{"status": "ok"}`, a `/docs` prikazuje popis ruta. Ta ista ruta služi i u produkciji: platforma njome provjerava je li primjerak aplikacije spreman primiti promet.

Dijagnostika i kvaliteta

Zapisnik rada, dosljedan oblik pogreške, statička analiza i formatiranje.

Što ulazi u zapisnik

Aplikacija usput zapisuje što je radila. Taj zapisnik (engl. *log*) jedini je trag kojim se poslije rekonstruira što se dogodilo.

Zapisuje se

ruta, metoda i statusni kod odgovora

ispis iznimke s popisom poziva, u razvojnoj okolini

domenski događaji, primjerice odjava natjecatelja

Ne zapisuje se

lozinke i tokeni, ni u dijelovima

tijelo zahtjeva s osobnim podacima

podatci po kojima se osoba može identificirati

Postavljanje zapisivanja

```
# app/core/logging.py
import logging

def setup_logging(debug: bool = False) -> None:
    logging.basicConfig(
        level=logging.DEBUG if debug else logging.INFO,
        format="%(asctime)s %(levelname)s %(name)s: %(message)s",
    )

# in a service module:
logger = logging.getLogger(__name__)
logger.info("club created: id=%s", club.id)
```

logger se dohvaća preko `__name__`, pa svaki redak nosi naziv modula iz kojega dolazi. Zapis ide na standardni izlaz, odakle ga preuzima okolina u kojoj proces radi.

Dosljedan oblik pogreške

Svaki neuspjeh vraća tijelo istoga oblika, pa ga frontend obrađuje na jednome mjestu:

```
{  
  "code": "registration_closed",  
  "message": "Rok za prijave je istekao."  
}
```

- code je stalan i namijenjen kodu koji odlučuje što učiniti.
- message je namijenjen čovjeku i smije se mijenjati.

Iznimka koju podiže sloj services nosi oba polja i statusni kod. Jedan handler registriran na razini aplikacije hvata sve takve iznimke i od njih sastavlja odgovor, pa se to ne ponavlja ni u jednoj ruti.

Konvencija

Uz ta dva polja često dolazi i treće, s popisom polja koja nisu prošla provjeru. Potrebno je kad sučelje poruku prikazuje uz svaki unos u obrascu, a ne samo jednu na vrhu.

Što se vidi bez pokretanja koda

- **Promjena koja to nije.** Dva autora imaju različito postavljene uređivače: jedan uvlači s četiri razmaka, drugi s dva i pri spremanju preslaguje uvoze. Kad drugi otvori i spremi tuđu datoteku, `git diff` pokazuje četrdeset izmijenjenih redaka, a ponašanje je isto. Stvarne promjene treba tražiti među njima.
- **Pogreška vidljiva iz samoga koda.** Neiskorišten uvoz, varijabla kojoj je vrijednost dodijeljena a nikad pročitana, grana do koje izvođenje ne može doći, krivo napisan naziv.

Prvo uklanjanje alat koji kod zapisuje u jedan stil, drugo pronalazi alat koji ga pregledava. Ruff radi oboje: `ruff format .` i `ruff check ..`

Oba se pokreću lokalno i pri svakoj promjeni u repozitoriju. Ono što alat može provjeriti ne treba provjeravati čovjek.

Rad s verzijama koda

Commit kao zapis jedne promjene, čitanje razlike, naredbe za pregled povijesti.

Commit kao zapis jedne promjene

Povijest repozitorija je zapis o tome kako je aplikacija nastala. Koliko je taj zapis upotrebljiv, ovisi o tome koliko je promjena stavljeno u jedan commit.

- Commit koji dira jednu temu može se pročitati u cijelosti i, ako zatreba, poništiti sam za sebe.
- Commit s petnaest datoteka i tri nepovezane promjene ne može ni jedno ni drugo.
- Poruka opisuje namjeru (`add config loading from environment`), a ne postupak (`izmjene`).

Kad se pojavi pogreška koje prije nije bilo, pretraga povijesti pokazuje u kojem se commitu ponašanje promijenilo. Taj je odgovor koristan samo ako commit sadrži jednu promjenu.

Kako čitati razliku

- 1 Poruka commita: koji je cilj promjene.
- 2 Popis datoteka: koje su nove, a koje izmijenjene.
- 3 Promjene vidljive prema van: nova ruta, promijenjena shema, novo polje u odgovoru.
- 4 Promjene u konfiguraciji: varijable okoline, opis usluga, zavisnosti.
- 5 Tek na kraju pojedinosti izvedbe.

Redoslijed je namjeran: prva tri koraka kažu mijenja li se ugovor prema klijentu, a to je jedino što odmah utječe na frontend.

Korisne naredbe

```
# history, one line per commit
git log --oneline --decorate

# what a single commit changed
git show <commit>

# difference between two commits
git diff <commit1>..<commit2>

# uncommitted changes in the working tree
git diff
```

Zadnja naredba pokazuje što je promijenjeno, a još nije zabilježeno. Pregled te razlike prije svakog commita hvata zaboravljene ispise, privremene izmjene i datoteke koje ne bi trebale ući u repozitorij.

Sažetak

- Backend prima HTTP zahtjev i sastavlja odgovor. U najmanjem obliku to je funkcija pridružena putanji.
- Čim podatci nisu javni i čim se mijenjaju, uz to dolaze utvrđivanje pošiljatelja, provjera prava, provjera oblika poslanih podataka, poslovna pravila i transakcija.
- Provjera koja pripada većem broju ruta zapisuje se jednom, u sloju services, i odatle se poziva.
- Statusni kod nosi značenje: 401 je neutvrđen identitet, 403 nedostatak ovlasti, 409 sukob sa stanjem resursa. Za neispravne podatke projekt bira između 400 i 422 i toga se drži.
- Konfiguracija dolazi iz varijabli okoline, pa prijelaz u produkciju ne mijenja kod; .env ostaje izvan repozitorija.

Što slijedi

- **15.10.2026.** Predavanje 2, modeliranje podataka: entiteti Klub i Korisnik, veza s bazom preko ORM-a, migracije alatom Alembic i početno punjenje baze.
- **22.10.2026.** Predavanje 3, autentikacija i sigurnost.
- **19.11.2026.** Obrana prvoga dijela projekta.

Do sljedećega predavanja: pokrenuti bazu i backend na vlastitom računalu te potvrditi odgovor na ruti `/health`.